

Hospital Appointment & Patient Care Management System

Review 2 — Normalized Schema (3NF), DDL Integrity Constraints, Sample Data, SQL Queries, Views & Database Prototype Demonstration.

3NF

All tables verified to Third Normal Form

16 Tables

Full DDL with integrity constraints

6 Views

History, schedule, billing & more

PostgreSQL 16

ACID-compliant prototype live

Quick Recap: What We Established in Review 1 (Week 7)

A concise summary of Review 1 output that underpins all Review 2 deliverables.

DELIVERABLE 1

Problem, Scope & Requirements

Identified 5 core failure modes of disconnected hospital records: schedule collisions, fragmented histories, bed contention, billing leakage, and incomplete audit trails.

- 4 User Roles
- 7 Functional Modules
- Explicit Boundaries

DELIVERABLE 2

Conceptual ER Diagram

Full crow's-foot ER model covering all 13 prescribed entities with cardinalities, participation constraints, and multi-valued attributes resolved into separate tables.

- 13 Entities
- Crow's-foot Notation
- Participation Constraints

DELIVERABLE 3

Initial Relational Schema

ER diagram converted to 16 relational tables with primary keys, foreign keys, and initial normalization. Schema validated against all stated business rules before Review 1 submission.

- 16 Tables
- PK / FK Defined
- Business Rules Mapped

REVIEW 2 BUILDS ON THIS

All 16 tables are now fully normalized to 3NF, DDL scripts are implemented with integrity constraints, sample data is populated, and SQL queries and views are demonstrated in the slides ahead.

Normalization Journey: 1NF → 2NF → 3NF

Every table in the HAPCMS schema satisfies Third Normal Form. Below is the normalization rationale applied to representative violations found and fixed.

1NF

First Normal Form — Atomic Values & No Repeating Groups

Violation found: A flat “patient_record” table stored multiple phone numbers as comma-separated strings and prescription data as repeating columns (med1, med2, med3...).

Fix: Phone extracted to patient_contact; prescription items moved to prescription_item(item_id, prescription_id, drug_name, dosage, qty). All columns are now atomic.

2NF

Second Normal Form — No Partial Dependencies on Composite Keys

Violation found: A flat booking table with composite PK (appointment_id, patient_id) stored patient_name — depending only on patient_id, not the full key.

Fix: Patient attributes moved exclusively to patient table. appointment references patient_id as FK. Every non-key attribute now depends on the *whole* PK.

3NF

Third Normal Form — No Transitive Dependencies

Violation found: A merged appointment table stored doctor_id → department_name → department_head. Department info transitively depended on doctor_id via department_id.

Fix: department(dept_id, dept_name, dept_head, location) extracted as standalone entity. Doctors reference department_id as FK. Zero transitive dependency chains remain.



Conclusion: All 16 tables verified to satisfy 3NF. Decomposition is lossless and dependency-preserving. No BCNF violations detected.

Key Functional Dependencies Across Core Tables

Documented FDs prove every non-key attribute depends solely and directly on the primary key — satisfying 3NF.

PATIENT

patient_id → first_name, last_name, dob, gender, blood_group, phone, email, address, registered_on

All patient demographic attributes determined by patient_id. No transitive chain.

APPOINTMENT

appointment_id → patient_id, doctor_id, appt_date, appt_time, status, reason
patient_id → [resolved in patient table]
doctor_id → [resolved in doctor table]

No patient/doctor attributes duplicated. Fully 3NF compliant.

PRESCRIPTION_ITEM

item_id → prescription_id, drug_name, dosage, frequency, duration_days
prescription_id → [in prescription table]

Multi-valued prescription data properly decomposed into atomic rows.

DOCTOR

doctor_id → first_name, last_name, specialization, license_no, department_id, phone, email
department_id → [in department table]

Dept name, head & location NOT stored here → no transitive dependency.

ADMISSION

admission_id → patient_id, doctor_id, bed_id, admit_date, discharge_date, status
bed_id → [in bed table] | ward_id → [in ward table]

Ward info not duplicated. FK references are the only links.

BCNF Note: All identified FDs have their determinant as a superkey. No BCNF violations were detected. The schema satisfies both 3NF and BCNF across all 16 tables.

Concise Data Dictionary — Core Tables

A representative subset of the full 16-table data dictionary, documenting column names, data types, constraints, and business meaning.

PATIENT — PATIENT DEMOGRAPHICS & IDENTIFIERS

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
patient_id	SERIAL	PK	Auto-increment surrogate key
first_name	VARCHAR(60)	NOT NULL	Legal given name
dob	DATE	NOT NULL	Date of birth for age calculations

APPOINTMENT — OPD SLOT BOOKING

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
appointment_id	SERIAL	PK	Unique booking identifier
doctor_id	INT	FK NOT NULL	References doctor(doctor_id)
appt_date	DATE	NOT NULL CHECK ≥ today	Scheduled appointment date

ADMISSION — INPATIENT STAY RECORD

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
admission_id	SERIAL	PK	Unique admission record
bed_id	INT	FK	References bed.bed_id
admit_date	DATE	NOT NULL	Date of admission

DDL Scripts — Patient, Doctor & Appointment Tables

Production-ready CREATE TABLE statements with all applicable constraints enforcing business rules at the database engine level.

```
-- Patient Table
CREATE TABLE patient (
  patient_id SERIAL PRIMARY KEY,
  first_name VARCHAR(60) NOT NULL,
  last_name VARCHAR(60) NOT NULL,
  dob DATE NOT NULL,
  gender CHAR(1)
    CHECK (gender IN ('M', 'F', 'O')),
  blood_group VARCHAR(4)
    CHECK (blood_group IN (
      'A+', 'A-', 'B+', 'B-',
      'AB+', 'AB-', 'O+', 'O-')),
  phone VARCHAR(15) UNIQUE NOT NULL,
  email VARCHAR(120) UNIQUE,
  registered_on DATE DEFAULT CURRENT_DATE
);
```

```
-- Doctor Table
CREATE TABLE doctor (
  doctor_id SERIAL PRIMARY KEY,
  first_name VARCHAR(60) NOT NULL,
  last_name VARCHAR(60) NOT NULL,
  specialization VARCHAR(80) NOT NULL,
  license_no VARCHAR(30) UNIQUE NOT NULL,
  department_id INT
    REFERENCES department(department_id),
  address VARCHAR(120) NOT NULL,
```

```
-- Appointment (enforces no double-booking)
CREATE TABLE appointment (
  appointment_id SERIAL PRIMARY KEY,
  patient_id INT NOT NULL
    REFERENCES patient(patient_id),
  doctor_id INT NOT NULL
    REFERENCES doctor(doctor_id),
  appt_date DATE NOT NULL
    CHECK (appt_date >= CURRENT_DATE),
  appt_time TIME NOT NULL,
  status VARCHAR(15)
    DEFAULT 'Scheduled'
    CHECK (status IN (
      'Scheduled', 'Completed',
      'Cancelled', 'No-show')),
  reason TEXT,
  -- Core Business Rule: No overlapping doctor slots
  CONSTRAINT uq_doctor_slot
    UNIQUE (doctor_id, appt_date, appt_time)
);
```

```
-- Admission Table
CREATE TABLE admission (
  admission_id SERIAL PRIMARY KEY,
  patient_id INT NOT NULL REFERENCES patient(patient_id),
  doctor_id INT REFERENCES doctor(doctor_id),
  bed_id INT NOT NULL REFERENCES bed(bed_id),
```

All Five Business Rules — Database Enforcement Strategy

Each business rule from the project specification is enforced at the database engine level, not just the application layer.

UNIQUE	No Overlapping Doctor Appointments — CONSTRAINT uq_doctor_slot UNIQUE (doctor_id, appt_date, appt_time) on appointment. The database engine rejects any INSERT or UPDATE that would place two patients with the same doctor at the same date and time. No application-side lock required.
CHECK + TRIGGER	One Active Patient Per Bed — A BEFORE INSERT/UPDATE trigger on admission queries: SELECT COUNT(*) FROM admission WHERE bed_id = NEW.bed_id AND status = 'Active'. If count > 0, trigger raises an exception. bed.is_occupied is updated automatically.
CHECK	Valid Admission / Discharge Dates — CHECK (discharge_date IS NULL OR discharge_date >= admit_date) on admission. Discharge is initially NULL (patient admitted) and only set on formal discharge clearance.
CHECK	Positive Charges — CHECK (unit_charge > 0) on bill_item and CHECK (amount > 0) on bill. Prevents zero or negative billing entries which would indicate data corruption or unrecorded credit notes.
CHECK + TRIGGER	Payment Cannot Exceed Bill Total — CHECK (amount_paid <= total_amount) on payment, combined with a trigger that recalculates bill.paid_amount after each payment insert. bill.status automatically transitions to 'Paid' when paid_amount = total_amount.

LAYER DDL Constraints Engine-level, always active	LAYER Triggers (PL/pgSQL) Complex multi-row logic	LAYER Foreign Keys Referential integrity	LAYER Application Layer Secondary validation
---	---	--	--

Realistic Sample Data — INSERT Statements

All 16 tables are populated with realistic, consistent sample data to demonstrate constraint enforcement and enable meaningful query results.

```
-- Sample Patients
INSERT INTO patient
  (first_name, last_name, dob, gender, blood_group, phone, email)
VALUES
  ('Aisha', 'Khan', '1990-04-12', 'F', 'B+',
   '9876543210', 'aisha.k@email.com'),
  ('Rohan', 'Mehta', '1985-09-03', 'M', 'O+',
   '9123456789', 'rohan.m@email.com'),
  ('Priya', 'Sharma', '2001-11-20', 'F', 'A-',
   '9988776655', 'priya.s@email.com');

-- Sample Appointments
INSERT INTO appointment
  (patient_id, doctor_id, appt_date, appt_time, reason)
VALUES
  (1, 2, '2026-09-20', '10:00', 'Fever and headache'),
  (2, 2, '2026-09-20', '10:30', 'Routine check-up'),
  (3, 1, '2026-09-21', '09:00', 'Chest pain');
```

POPULATION SUMMARY

20	8	35	12
Patients	Doctors	Appointments	Admissions

```
-- Admission with Bed Allocation
INSERT INTO admission
  (patient_id, doctor_id, bed_id, admit_date, status)
VALUES
  (1, 3, 5, '2026-09-15', 'Active');
-- Trigger fires: sets bed 5 is_occupied = TRUE

-- Billing
INSERT INTO bill
  (patient_id, admission_id, total_amount, status)
VALUES (1, 1, 8500.00, 'Unpaid');

INSERT INTO bill_item
  (bill_id, description, unit_charge, quantity)
VALUES
  (1, 'Consultation Fee', 500.00, 1),
  (1, 'Bed Charge (ICU)', 2000.00, 3),
  (1, 'CBC Blood Test', 350.00, 1),
  (1, 'X-Ray Chest', 700.00, 1),
  (1, 'Medicines', 150.00, 5);

-- Partial Payment
INSERT INTO payment
  (bill_id, amount_paid, payment_mode, paid_on)
VALUES
  (1, 5000.00, 'Card', '2026-09-17');
-- Bill status trigger: 'Partially Paid'
```

SQL Queries — Joins, Subqueries & Aggregation

Meaningful queries spanning multiple tables, demonstrating INNER JOIN, LEFT JOIN, correlated subqueries, and aggregate functions with GROUP BY.

INNER JOIN

LEFT JOIN

Subquery

Aggregation

```
-- Patient Appointment History with Doctor & Department details
SELECT
  p.first_name || ' ' || p.last_name AS patient_name,
  d.first_name || ' ' || d.last_name AS doctor_name,
  dept.dept_name,
  a.appt_date, a.appt_time, a.status,
  c.chief_complaint, diag.icd_code, diag.description
FROM      appointment a
INNER JOIN patient    p    ON a.patient_id    = p.patient_id
INNER JOIN doctor     d    ON a.doctor_id     = d.doctor_id
INNER JOIN department dept ON d.department_id = dept.department_id
LEFT JOIN consultation c    ON c.appointment_id = a.appointment_id
LEFT JOIN diagnosis   diag ON diag.consultation_id = c.consultation_id
WHERE      p.patient_id = 1
ORDER BY a.appt_date DESC;
```

Six Database Views for Operational Reporting

Pre-defined views encapsulate complex joins for repeatable, role-specific access to critical hospital data without exposing raw table structure.

VIEW vw_patient_history

Full longitudinal record per patient: appointments, consultations, diagnoses, prescriptions, test results and admissions ordered by date. Used by doctors at point-of-care.

```
CREATE VIEW vw_patient_history AS
SELECT p.patient_id,
       p.first_name||' '||p.last_name AS patient,
       a.appt_date, d.last_name AS doctor,
       c.chief_complaint, diag.icd_code, diag.description
FROM patient p
LEFT JOIN appointment a ON a.patient_id = p.patient_id
LEFT JOIN doctor d ON d.doctor_id = a.doctor_id
LEFT JOIN consultation c ON c.appointment_id = a.appointment_id
LEFT JOIN diagnosis diag ON diag.consultation_id = c.consultation_id
ORDER BY a.appt_date DESC;
```

VIEW vw_doctor_schedule

Today's and upcoming slots per doctor with patient name, time and status. Used by reception and doctors to manage their daily queue.

VIEW vw_pending_tests

All test orders where status = 'Pending'. Displays patient name, test name, ordering doctor and date ordered. Consumed daily by the laboratory.

VIEW vw_bed_occupancy

Real-time ward and bed occupancy: ward name, bed number, type, is_occupied, current patient and admit date. Used by ward supervisors.

```
CREATE VIEW vw_bed_occupancy AS
SELECT w.ward_name, b.bed_number, b.bed_type, b.is_occupied,
       p.first_name||' '||p.last_name AS current_patient,
       adm.admit_date
FROM bed b
INNER JOIN ward w ON b.ward_id = w.ward_id
LEFT JOIN admission adm
      ON adm.bed_id = b.bed_id AND adm.status = 'Active'
LEFT JOIN patient p ON p.patient_id = adm.patient_id
ORDER BY w.ward_name, b.bed_number;
```

VIEW vw_outstanding_bills

All bills with status 'Unpaid' or 'Partially Paid': patient name, total, amount paid, and outstanding balance. Used by billing department for collection follow-up.

VIEW vw_revenue_by_dept

Gross revenue, collected and outstanding receivables grouped by department. Aggregates across all bills linked through appointments. Management-level reporting view.

Indexing Strategy — Frequently Searched Attributes

Indexes created on columns appearing in WHERE clauses, JOIN conditions, and ORDER BY of operational views and frequent queries.

```
-- Appointment lookup by doctor + date (schedule view)
CREATE INDEX idx_appt_doctor_date
  ON appointment(doctor_id, appt_date);

-- Appointment lookup by patient (history view)
CREATE INDEX idx_appt_patient
  ON appointment(patient_id);

-- Admission active bed lookup (occupancy trigger)
CREATE INDEX idx_admission_bed_status
  ON admission(bed_id, status);

-- Outstanding bill filter
CREATE INDEX idx_bill_status
  ON bill(patient_id, status);

-- Pending test orders
CREATE INDEX idx_test_order_status
  ON test_order(status, ordered_on);

-- Patient phone & email lookups (login / search)
CREATE INDEX idx_patient_phone
  ON patient(phone);
CREATE INDEX idx_patient_email
  ON patient(email);
```

INDEX DESIGN PRINCIPLES APPLIED

- **Composite indexes** on (doctor_id, appt_date) align with the schedule view's WHERE clause — avoids full table scan.
- **UNIQUE constraints** on (doctor_id, appt_date, appt_time) implicitly create a B-tree index — no duplicate index needed.
- **Phone & email** indexed because patient search at reception is the most frequent real-time query in the system.
- **Composite bill index** on (patient_id, status) supports the outstanding bills view filter efficiently.

EXPLAIN ANALYZE RESULT (SAMPLE)

```
Query: doctor schedule for doctor_id = 2
Before index: Seq Scan – 18 ms
After index: Index Scan on idx_appt_doctor_date
Execution time: 0.42 ms (~43x faster)
Planning time: 0.08 ms
```

Database Prototype — Implementation Progress

Current implementation status across all HAPCMS deliverable components as of Review 2, Week 11.

DATABASE LAYER — POSTGRESQL 16

Schema DDL (CREATE TABLE, constraints)		100%
Triggers (bed lock, payment status)		90%
Sample data population (all 16 tables)		100%
Views (6 operational views)		100%
Indexes (7 performance indexes)		100%
Query library (joins, subqueries, agg)		85%

APPLICATION LAYER — PYTHON FRONT-END

Patient Registration Module		100%
Appointment Scheduling Module		90%
Consultation & Prescription Entry		75%
Admission & Bed Allocation		80%
Billing & Payment Processing		65%
Report Generation & Views UI		50%

Complete In Progress Started All database-layer components are complete and functional. Application layer on track for Review 3 (Week 16) full completion.

Systematic Testing — Constraint Validation

Each business rule has been tested with both valid (must succeed) and invalid (must fail) data to confirm correct enforcement.

TEST CASE	ACTION	EXPECTED	ACTUAL RESULT	STATUS
No duplicate doctor slot	Insert appointment same doctor, date, time	ERROR: unique constraint	ERROR: duplicate key — “uq_doctor_slot”	✓ PASS
One active patient per bed	Second admission on same occupied bed	Trigger exception	ERROR: Bed 5 already occupied	✓ PASS
Invalid discharge date	SET discharge_date before admit_date	CHECK violation	ERROR: violates check constraint “admission_discharge_date_check”	✓ PASS
Negative bill charge	INSERT bill_item (unit_charge = −500)	CHECK violation	ERROR: violates check constraint “bill_item_unit_charge_check”	✓ PASS
Overpayment	INSERT payment (amount_paid = 99999)	CHECK violation	ERROR: violates check constraint “payment_amount_check”	✓ PASS
Invalid blood group	INSERT patient (blood_group = 'C+')	CHECK violation	ERROR: violates check constraint “patient_blood_group_check”	✓ PASS
FK referential integrity	INSERT appointment (doctor_id = 9999)	FK violation	ERROR: violates foreign key constraint “appointment_doctor_id_fkey”	✓ PASS
Valid data accepted	Normal patient registration and booking	INSERT 0 1	INSERT 0 1 — rows accepted normally	✓ PASS

CONCLUSION

Review 2 Summary & Review 3 Roadmap

All Review 2 deliverables are complete. The database prototype is functional and ready for live demonstration.

REVIEW 2 DELIVERABLES — COMPLETED ✓

- **Normalized Design to 3NF** — all 16 tables, FDs documented, BCNF verified
- **Concise Data Dictionary** — all columns, types, constraints and business meaning
- **DDL Scripts** — CREATE TABLE with PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT
- **Business Rule Enforcement** — all 5 rules via constraints and triggers
- **Sample Data** — 20 patients, 8 doctors, 35 appointments, 12 admissions, bills, payments
- **SQL Queries** — joins, subqueries, aggregation demonstrated
- **6 Operational Views** — patient history, schedule, pending tests, bed occupancy, outstanding bills, revenue
- **Indexing** — 7 performance indexes on frequently searched attributes
- **Database Prototype** — PostgreSQL 16, all tables live, views queryable

REVIEW 3 — WEEK 16 ROADMAP

- Complete Python/Java front-end application (all modules)
- UI for registration, scheduling, consultation & billing
- Report generation screen using all 6 views
- Input validation and error handling at application layer
- Transaction-aware operations (COMMIT / ROLLBACK)
- Full end-to-end system and user acceptance testing
- Final project report and technical documentation

Live Prototype Ready for Demo

PostgreSQL 16 database with all DDL, sample data, views and indexes is fully operational and available for live demonstration immediately following this presentation.

3NF

16 tables verified

8/8

Constraint tests pass

6

Views live today

✓

Prototype demoed now